COM2041 Assignment
Spring Semester 2026
Prof Steve Schneider

Due Wednesday 18th March 4pm
Returned by Friday 27th March

Submission through SurreyLearn

This coursework tests your understanding of various cryptographic schemes covered in the lectures, and adaptations of them.

There are 8 questions. You will be provided with your individual instances of the problems (different for each student).

You are free to use any method you wish for solving the problems. You can use online tools, you can write code, you can do it by hand, you can make use of AI tools, or use any other method you think of.

However, you may not use any custom programs or code written by someone else in the class or downloaded from the Internet.

You are expected to spend 20-25 hours on this coursework.

Note that you have been given individual ciphertexts and hash functions specific to you. These can be found under "Individual Values" under Assignment Resources in SurreyLearn. Everyone has a different problem set. This is to prevent copying of other people's work.

To submit your solutions you should enter your answers to each question into the **Submission Generator** within Assignment Resources. This will generate a .txt file. **You should then upload this file as your coursework submission.**

Hints and tips

1. *Note that the questions are of varying levels of difficulty*. If you really cannot see how to do a question and have already spent a substantial amount of time on it, then move on to the next one.

2. Start in good time: don't leave it until the day before it is due in.

3. **THINK** about a question before you attempt it, and think about how your understanding of the topics in the course can be applied.

4. Use the COM2041 discussion board to raise and answer questions about the assignment.

5. Various helpful files are provided in SurreyLearn, including **ukenglish**, **10-letter-words** and **11-letter-words**.

Questions

1. Caesar Cipher [10%]

Your task is to decrypt a ciphertext encrypted with a Caesar cipher.

For the Caesar Ciphertext you have been given:

a) Give the key used to produce the ciphertext
b) Give the Plaintext (with correct upper/lower case and punctuation).

2. CaesarPlus [10%]

This question introduces a cryptosystem which is a more complex version of Caesar, called **CaesarPlus.** To encrypt with CaesarPlus, the key consists of a character *k* and also a triple of values [*a, b, c*]

To encrypt a plaintext character p, CaesarPlus returns a ciphertext character obtained by calculating:

$$a.p^3 + b.p^2 + c.p + k \mod 26$$

To encrypt a string, each letter of the plaintext in turn is encrypted using [a,b,c] and k. For example, if a=1, b=4, c=7, k = 'f', then the encryption of the character **C** (value 2) will be

$1.2^3 + 4.2^2 + 7.2 + 5 \mod 26 = 8+16+14+5 \mod 26 = 43 \mod 26 = 17 =$ **R**

Note that if a=b=0 and c=1 then CaesarPlus is the same as the Caesar Cipher.

You are given a ciphertext encrypted under CaesarPlus, with a>0 and b>0.

a) **Give the triple of values *[a, b, c]***
b) **Give the key k**
c) **Give the plaintext (with correct upper/lower case and punctuation).**

Notice that not all sets of values for [a,b,c] give a cipher scheme (i.e. a permutation of the alphabet). For instance, using the example above, encrypting the character **B** (value 1) gives

$1.1^3 + 4.1^2 + 7.1 + 5 \mod 26 = 1+4+7+5 \mod 26 = 17 \mod 26 = 17 =$ **R**

In this case both plaintexts **B** and **C** encrypt to the same cipher character **R.** Therefore CeasarPlus with [1,4,7] and 'f' does not give a cipher scheme.

CONTINUED NEXT PAGE

3. CaesarStream [10%]

CaesarStream is another variation of the Caesar cipher. Encryption in CaesarStream takes an Initialisation Character **i** and a key character **k**. We convert a plaintext P to a ciphertext C, where P(j) is the jth character of P and C(j) is the jth character of C.

The first ciphertext character C(1) is obtained as the plaintext character offset with both the key and the initialisation character. Ciphertext character C(j+1) is given as the plaintext character P(j+1) offset with both the key and ciphertext character C(j):

$$C(1) = P(1) + k + i$$

$$C(j+1) = P(j+1) + k + C(j)$$

All addition is modulo 26 similar to Caesar Cipher.

You have been given a ciphertext consisting of a plaintext encrypted under CaesarStream, with the spaces and punctuation removed.

a) **Give the key k**
b) **Give the initialization character i**
c) **Give the plaintext (marks awarded for correct characters and spaces; punctuation does not need to be reconstructed)**

4. Password Cracking [10%]

A system requires users to enter a password to obtain access. A system protected by password generates a sequence of random characters, known as *salt*, appends the salt to the password, and stores the hash of the result together with the salt. When a password is submitted then it appends the salt, computes the hash, and compares with the hash value that is stored.

For example, if the password is **contemplates** and the salt is **FZ23sDK0NcUi** then the hash value will be:

7a8a1ef53704c6ba6bee1c0ac8a99c74b17d59228297d6d877a36b9cb836221e

Sandy has decided to use a word from the dictionary as a password. You are given the SHA-256 hash value and the salt for this password.

a) **Give the password**

CONTINUED NEXT PAGE

5. Password Cracking on trickier password [10%]

Sam and Alex have a dog whose name is one of the top 50 most popular dog's names, given in https://poppyspicnic.co.uk/blogs/raw-pet-food/top-50-most-popular-dog-names-in-the-uk-2025-edition.

They are keen to use a password which they can remember but which is difficult to crack. They read of a technique in the *Daily Rag* of using some personal information that no-one would ever guess, and then cleverly changing some of the characters to numbers or symbols, and also appending some digits. They have taken this approach as follows:

1. They start with their dog's name
2. They randomly make some substitutions: letters might be left alone, or might be substituted as follows: o/O becomes 0 or *; i/I becomes 1 or !; l/L becomes 1; a/A becomes 4, @ or &; e/E becomes 3; s/S becomes $ or 5.
3. They add two random digits at the end.

For example, "Alfie" may be transformed to "4lfi318"

Unfortunately the salt, and the hash value of their password with salt appended, have been leaked in a data breach, and their password might be vulnerable. You are given the salt and the hash value.

a) Give the Password

b) Give the Dog's name as given on the list of popular dog names

6. Cracking a ciphertext with partially leaked key [10%]

An 11 letter plaintext word has been encrypted with a Vigenere key which is also of length 11, to give an 11-letter ciphertext. Some of the key characters have been leaked, but some of the characters are not yet known. You are given the partial key, and you are also given the ciphertext. Your task is to reconstruct the key and find the plaintext.

a) Give the full key
b) Give the plaintext

7. Cracking reused one-time pads [10%]

Charlie wants to encrypt some 10 letter words, and is aware that using a 10-letter Vigenere key just once is equivalent to using a one-time pad and hence produces an uncrackable ciphertext.

However Charlie is lazy and only produces one key, which is a string of 10 random characters. There are two 10-letter words to encrypt. Charlie thinks it will be fine to use the same key for both, and produces two ciphertexts encrypted with the same key.

You are provided with the two ciphertexts obtained from encrypting the two 10-letter words with a 10-letter Vigenere key.

a) Give the key k
b) Give each plaintext word

8. VigenerePlus [30%]

This question introduces a cryptosystem which is a more complex version of Vigenère, called **VigenerePlus.** To encrypt with VigenerePlus, you use a key as in Vigenere, but also associate **each letter of the key** with a **triple of multipliers,** as seen in Question 2 on CaesarPlus.

A plaintext is encrypted by repeating the key to the length of the plaintext, and then encrypting each letter in turn.

You have been given a ciphertext encrypted under VigenerePlus. The spaces and punctuation have been removed, and the characters have all been put into lower case. The keyword is a word from the dictionary, and the multiplier triples are randomly chosen. Your task is to reconstruct the plaintext. You are not required to give the plaintext but for your answer you should provide the following items:

a) **Give the keylength**
b) **Give the keyword**
c) **Give the list of corresponding multiplier triples as a comma-separated list of values, enclosed in square brackets, e.g. [[1,2,3],[4,5,6],[7,8,9]]**
d) **Give the surname of the author of the plaintext**